Property-Based Fuzzing of Solidity Smart Contracts with Echidna

Federico Antolini 1985581
Security in Software Applications – 2025/2026

1 Introduction

This report describes the property-based fuzzing of two Solidity smart contracts, `Taxpayer.sol` and `Lottery.sol`, using Echidna. The contracts model a simplified tax system: individuals can marry, pool part of their income tax allowance with their spouse, receive a higher allowance once they turn 65, and participate in a periodic commit-reveal lottery whose winner receives a further increase in allowance.

1.1 What fuzzing is, and how Echidna enforces properties

Fuzzing is a testing technique that feeds a program with large numbers of automatically generated inputs in order to trigger unexpected behaviour – crashes, failed assertions, or, more generally, violations of properties the program is expected to satisfy. Traditional fuzzers look for crashes; *property-based* fuzzers such as Echidna instead look for a violation of *invariants* explicitly written by the developer.

Concretely, Echidna takes a Solidity contract and treats every one of its public and external functions as a possible action. It then generates long, random sequences of calls to these functions – with randomly chosen arguments, senders and, to a lesser extent, timestamps – and after every single call it re-evaluates a set of user-defined *invariant functions*. In the default *property* testing mode (the one used throughout this project) an invariant is any function whose name starts with `echidna_`, takes no arguments, and returns a `bool`. As long as every invariant keeps returning `true`, the campaign continues. The moment one returns `false` (or reverts unexpectedly), Echidna has *falsified* that property, and it reports the shortest call sequence it could find that reproduces the failure – which is invaluable for understanding *why* the property broke. Echidna is coverage-guided: it keeps a corpus of interesting call sequences (ones that reached new code paths) and mutates them, rather than sampling purely at random, which makes it considerably more effective than naive random testing at reaching deep, multi-transaction bugs.

Because an invariant is just an ordinary Solidity function, it can call into the contract under test (or into helper/wrapper contracts I write myself) to inspect state and cross-check it against what *should* hold. This is what makes the technique powerful for contracts like these: a bug such as "a marriage is asymmetric" or "the sum of a couple's tax allowances changed after a legitimate transfer" is not something a single `require` inside the contract would normally catch – it only becomes visible when you compare the state of *two different contract instances* after an arbitrary sequence of calls, which is exactly what a dedicated invariant function can do.

1.2 Methodology

For every part of the assignment I followed the same iterative loop, applied one or more times depending on how many distinct issues Echidna surfaced:

1. Write invariants expressing the property required by the assignment, plus any additional property that seemed reasonable given the code (e.g. "nobody can be married to themselves").

2. Write a small Echidna harness contract that deploys a handful of `Taxpayer` (and, for Part 4, `Lottery`) instances and exposes thin wrapper functions Echidna can call with fuzzed arguments.

3. Run Echidna against the *current* version of the code (initially, the version handed out with the assignment) and inspect any `FAILED!` result, including the minimized call sequence and the execution trace.
4. Diagnose the root cause, fix it with the smallest reasonable change – a missing `require`, a missing state update, a wrong constant – and re-run.
5. Repeat until every invariant is `passing` for a full campaign (50 000 calls by default).

I refer to each iteration of this loop as a *round*. Some parts of the assignment needed a single round (the fix was correct on the first attempt); others needed two or three, when fixing one issue exposed – or simply didn't happen to cover – another. I report every round honestly, including a case where a fuzzing campaign passed a property purely because it hadn't yet tried the specific call sequence needed to break it (Part 4, `echidna_no_duplicate_reveals`), which I fixed anyway on code-review grounds. I believe this gives a more accurate picture of the process than presenting only a single, final, already-correct version of the code.

All Echidna runs shown in this report used Echidna 2.3.3 in the default *property* testing mode, on macOS, against `solc 0.8.22`.

2 Part 1 – Marriage symmetry

2.1 Property and invariants

The assignment requires that marriage be symmetric: if taxpayer x is married to y, then y must also be married to x. I expressed this, plus two additional properties I considered sensible, as three Echidna invariants inside a harness contract `TaxpayerEchidnaTest` that deploys three independent `Taxpayer` instances and exposes wrappers for `marry`, `divorce` and `haveBirthday`. Throughout this report I write calls in terms of these harness wrappers rather than the raw contract methods: e.g. `marry(i,j)` denotes the wrapper "make taxpayer i marry taxpayer j" (which internally calls `taxpayers[i].marry(address(taxpayers[j]))`), and indices are taken modulo the population size N. This is why the reported call sequences carry two integer arguments even though the underlying `marry()` takes a single address.

```
function echidna_marriage_is_symmetric() public view returns (bool) {
    for (uint i = 0; i < N; i++) {
        address spouseAddr = taxpayers[i].getSpouse();
        if (spouseAddr == address(0)) continue;
        bool foundInPopulation = false;
        for (uint j = 0; j < N; j++) {
            if (address(taxpayers[j]) == spouseAddr) {
                foundInPopulation = true;
                if (taxpayers[j].getSpouse() != address(taxpayers[i])) return
                    false;
                if (!taxpayers[j].getIsMarried()) return false;
            }
        }
        if (!foundInPopulation) return false;
    }
    return true;
}
```

For every taxpayer i that reports being married to someone, the invariant locates that spouse inside my small population and checks that the relation holds in the opposite direction too (`spouse` field *and* `isMarried` flag). I complemented this with `echidna_no_self_marriage` (nobody can be their own spouse) and `echidna_married_flag_consistent_with_spouse` (the boolean flag and the address field can never disagree with each other).

These invariants required adding read-only getters (`getSpouse`, `getIsMarried`) to `Taxpayer.sol`: the original contract exposed no way for an external contract to observe marital state at all, so without them Echidna could not check the property regardless of whether it held.

Against the original, unmodified `marry()`/`divorce()` – which only update the caller's own state – Echidna immediately falsified the property with a single call, `marry(1,0)`, and also found that nothing prevented self-marriage (`echidna_no_self_marriage` failed with `marry(0,0)`, disguised in the minimized trace as `marry(220221,0)` since indices are taken modulo the population size).

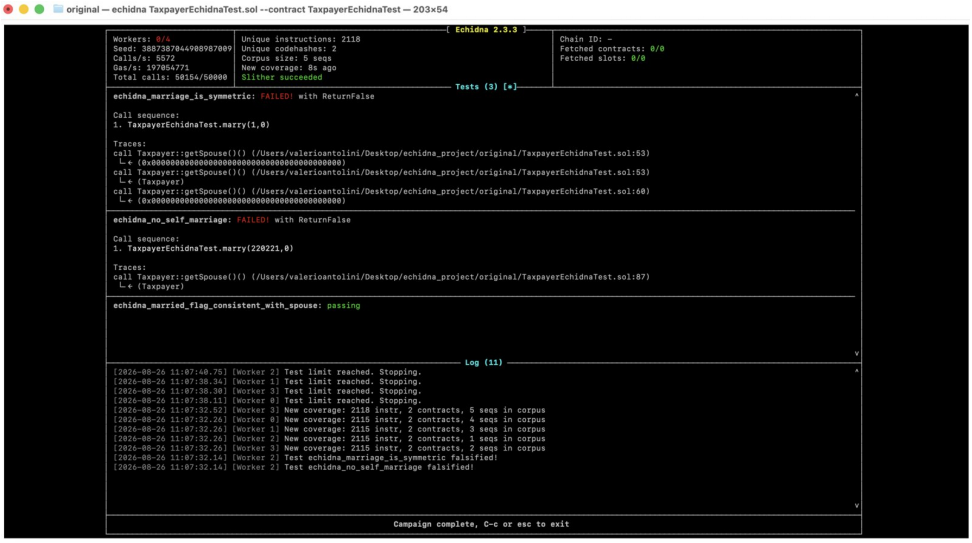


Figure 1: Echidna against the original `Taxpayer.sol`: marriage symmetry and self-marriage both falsified.

Fix. I changed `marry()` to validate its argument and to propagate the update to the other party in the same transaction, via a new helper `setSpouse()`:

```
function marry(address new_spouse) public {
  require(new_spouse != address(0), "cannot marry the zero address");
  require(new_spouse != address(this), "cannot marry yourself");
  spouse = new_spouse;
  isMarried = true;
  Taxpayer(new_spouse).setSpouse(address(this));
}
```

`divorce()` was fixed symmetrically, calling a new `clearSpouse()` helper on the ex-spouse's contract.

Re-running Echidna against this first fix, `echidna_marriage_is_symmetric` failed *again*, this time with a two-call sequence: `marry(0,1)` followed by `marry(2,0)`. Taxpayer 0, already married to taxpayer 1, was silently "stolen" by taxpayer 2's proposal – my fix propagated the update correctly, but never checked that either party was currently unmarried before overwriting the relationship. Taxpayer 1 was left believing it was still married to taxpayer 0, while taxpayer 0's spouse field had moved on to taxpayer 2: asymmetric again, for an entirely different reason.

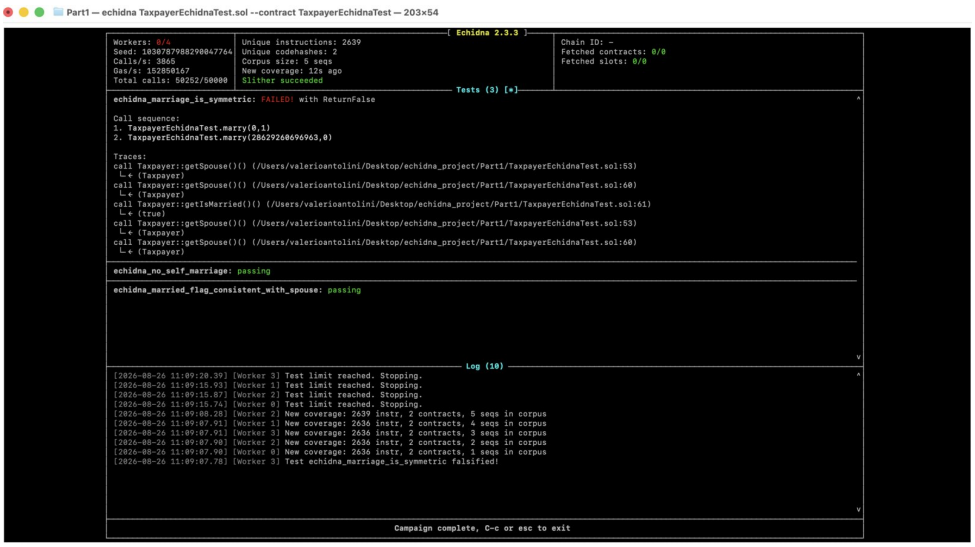


Figure 2: After the round 1 fix: a second, different bug breaks marriage symmetry again.

Fix. I added the missing precondition – the kind of fix explicitly anticipated in the assignment text ("adding a sensible precondition for the method, using `require()`") – on both sides of the marriage, and, defensively, inside `setSpouse()` itself, since it is a public function reachable directly and not only through `marry()`:

```
require(!isMarried, "you are already married");
require(!Taxpayer(new_spouse).getIsMarried(), "target is already married");
```

One helper, `setSpouse()`, deliberately keeps a weaker guard – `require(Taxpayer(msg.sender).isContract())`, plus the defensive `!isMarried` check shown above – rather than a full identity check. It is only ever meant to be invoked by the other `Taxpayer` from within `marry()`; hardening it into a proper two-step propose/accept handshake is a design change beyond the symmetry property tested here, and I return to this deliberately-retained weak point in the concluding section.

After this second fix, all three invariants passed a full 50 000-call campaign.

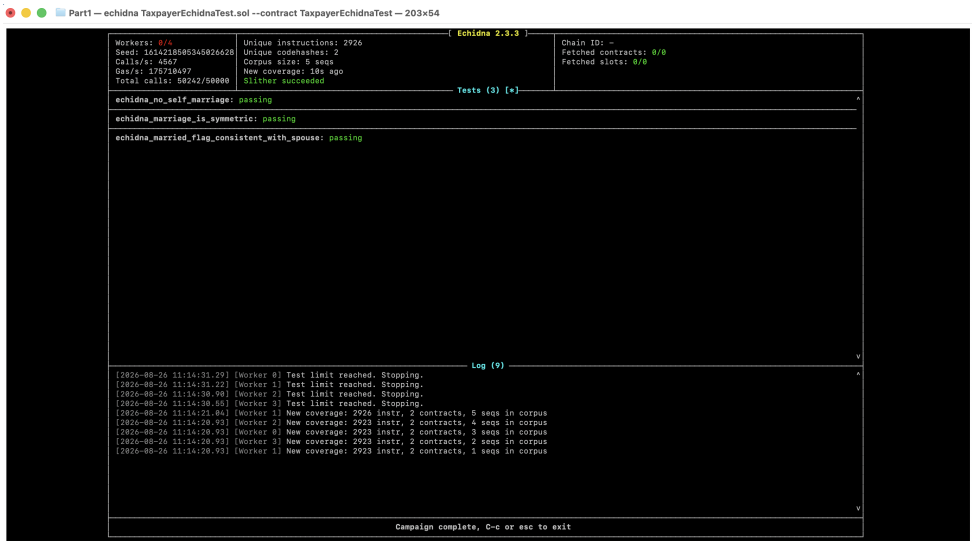


Figure 3: All three Part 1 invariants passing after both fixes.

3 Part 2 – Pooled tax allowance

3.1 Property and invariants

Married couples may redistribute their combined tax allowance between themselves as long as its *sum* stays constant. I extended the same harness with a wrapper for `transferAllowance` and two invariants: `echidna_pooled_allowance_sum_preserved`, checking that for every married pair the sum of the two allowances equals 2× the default (10 000), and `echidna_unmarried_has_default_allowance`, checking that a single taxpayer's allowance never drifts away from the default. On divorce, a couple loses this pooling feature and each spouse's allowance is reset to the individual default; this is implemented by resetting `tax_allowance` to the default inside both `divorce()` and `clearSpouse()`. The `transferAllowance` function itself carries two preconditions I added – `require(isMarried)`, since pooling only makes sense between spouses, and `require(change <= tax_allowance)`, since a taxpayer cannot give away more than it holds – which replace what would otherwise be confusing low-level reverts with explicit, meaningful failures.

```
function echidna_pooled_allowance_sum_preserved()
    public view returns (bool) {
  uint defaultAllowance = taxpayers[0].getDefaultAllowance();
  for (uint i = 0; i < N; i++) {
    if (!taxpayers[i].getIsMarried()) continue;
    address spouseAddr = taxpayers[i].getSpouse();
    for (uint j = 0; j < N; j++)
      if (address(taxpayers[j]) == spouseAddr && j > i) {
        uint sum = taxpayers[i].getTaxAllowance()
                 + taxpayers[j].getTaxAllowance();
        if (sum != 2 * defaultAllowance) return false;
      }
  }
  return true;
}
```

For every married pair – each counted once, thanks to the `j > i` guard, so a couple is never checked twice – the invariant reads both spouses' current allowances and requires their sum to equal twice the individual default (10 000). Pooling may therefore shift allowance from one spouse to the other in any proportion, but can never create or destroy it. The companion invariant `echidna_unmarried_has_default_allowance` is the mirror check for single taxpayers: anyone not currently married must sit exactly at the 5 000 default.

The original `setTaxAllowance()` only checks that the caller *is a contract* that reports `isContract() == true` – not that it is genuinely the taxpayer's spouse:

```
function setTaxAllowance(uint ta) public {
  require(Taxpayer(msg.sender).isContract() || Lottery(msg.sender).isContract
      ());
  tax_allowance = ta;
}
```

To demonstrate the consequence, I added a wrapper `maliciousSetAllowance` to the harness itself, together with an `isContract()` stub returning `true` – exactly what an attacker-deployed contract would need to satisfy the check above. A single call, `maliciousSetAllowance(0,0)`, immediately broke both invariants: any address that merely *looks like* a contract could rewrite anyone's allowance, married or not, entirely bypassing `transferAllowance()`.

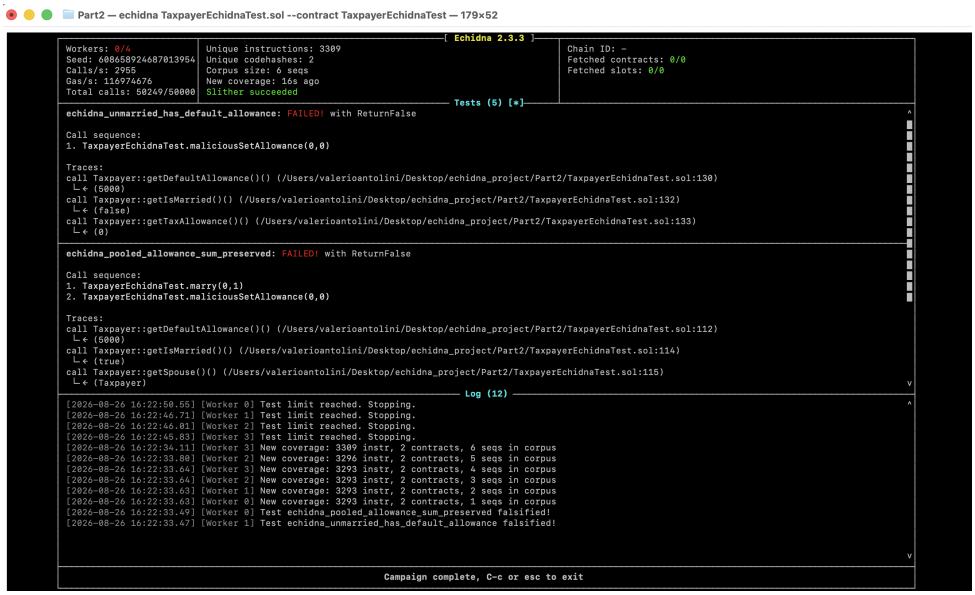


Figure 4: A simulated malicious contract rewrites another taxpayer's allowance in a single call.

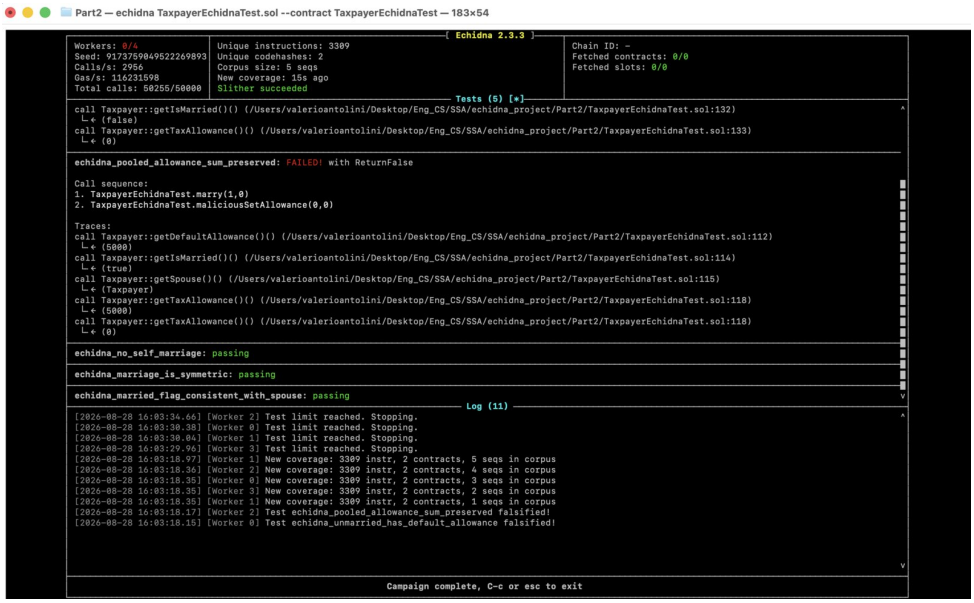


Figure 5: Full dashboard for the same campaign: the Part 1 regression invariants still pass, while both new Part 2 invariants are falsified.

Fix. I replaced the self-reported `isContract()` flag with a check against a real relationship. `Taxpayer` now remembers which `Lottery` it last joined, and `setTaxAllowance` only accepts calls from the taxpayer's actual current spouse or from that specific lottery contract:

```
function setTaxAllowance(uint ta) public {
  require(msg.sender == spouse || msg.sender == lastLottery,
          "not authorized to change this allowance");
  tax_allowance = ta;
}
```

After this fix, all five invariants (the three from Part 1, kept as regression checks, plus the two new ones) passed a full campaign.

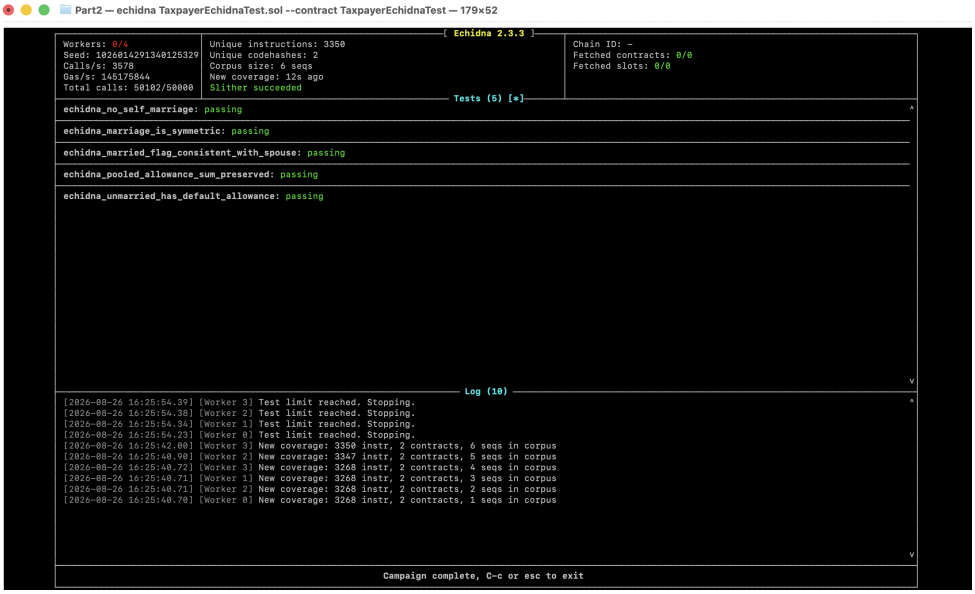


Figure 6: All five invariants passing after tightening `setTaxAllowance` authorization.

4 Part 3 – Age-dependent allowance

4.1 Property and changes

From age 65, a taxpayer's default allowance rises from 5 000 to 7 000, with the same pooling rule as before. This required three coordinated changes:

1. `haveBirthday()` now adds the delta (`ALLOWANCE_OAP - DEFAULT_ALLOWANCE`) to the *current* `tax_allowance` the exact moment `age` reaches 65 – rather than overwriting the value – so that any pooling adjustment already in place from `transferAllowance` is preserved:

```
function haveBirthday() public {
  age++;
  if (age == 65) {
    tax_allowance += (ALLOWANCE_OAP - DEFAULT_ALLOWANCE);
  }
}
```

2. `divorce()`/`clearSpouse()` now reset to an *age-appropriate* default via a small helper, `currentDefaultAllowance()`, instead of the flat constant used in Part 2.

3. `clearSpouse()`'s access control, previously the same weak `isContract()` pattern flagged in Part 1/2, was tightened to `require(msg.sender == spouse)` while I was already in that code, to avoid discovering the same class of bug a third time.

The two Part 2 invariants were generalised to compute the *expected* allowance from each taxpayer's current age rather than assuming a flat constant, and the harness gained a `haveManyBirthdays` wrapper (looping `haveBirthday` internally) so that Echidna can reach the 65-year threshold in a handful of transactions instead of dozens of consecutive individual calls.

```
function expectedAllowance(uint idx) internal view returns (uint) {
  uint age = taxpayers[idx].getAge();
  return age >= 65 ? taxpayers[idx].getOapAllowance()
                   : taxpayers[idx].getDefaultAllowance();
}

function echidna_pooled_allowance_sum_correct()
```

```
    public view returns (bool) {
  for (uint i = 0; i < N; i++) {
    if (!taxpayers[i].getIsMarried()) continue;
    address spouseAddr = taxpayers[i].getSpouse();
    for (uint j = 0; j < N; j++)
      if (address(taxpayers[j]) == spouseAddr && j > i) {
        uint sum = taxpayers[i].getTaxAllowance()
                 + taxpayers[j].getTaxAllowance();
        if (sum != expectedAllowance(i) + expectedAllowance(j))
          return false;
      }
  }
  return true;
}
```

The flat Part 2 target of $2 \times 5\,000$ is replaced by a per-taxpayer baseline computed by the helper `expectedAllowance`, which returns 7 000 once a taxpayer is 65 or older and 5 000 otherwise. The invariant then requires each married couple's combined allowance to equal the sum of their two *age-appropriate* baselines, so pooling stays sum-preserving even after one or both spouses cross 65. The single-taxpayer check `echidna_unmarried_allowance_correct` is generalised the same way.

Unlike Parts 1 and 2, this part passed a full campaign on the very first attempt.

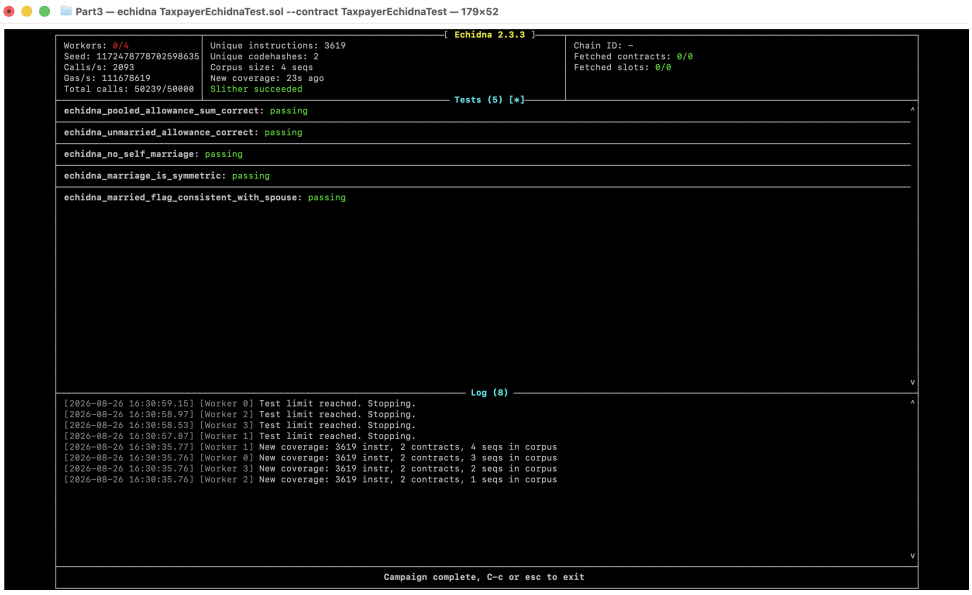


Figure 7: All six invariants passing on the first Part 3 campaign.

I attribute this to the narrower scope of the change: the two authorization weaknesses that had caused every previous failure were already closed by this point, and the new logic (an additive, one-shot update triggered by a simple equality check) has a much smaller state space than "anyone can rewrite anyone's marital or financial state". I do not take a single clean run as absolute proof of correctness – fuzzing gives evidence, not a guarantee – but combined with the manual reasoning above I consider it reasonably strong evidence for this part.

5 Part 4 – Commit-reveal lottery

This part covers `Lottery.sol`, a periodic commit-reveal lottery: a taxpayer under 65 commits $y = \text{keccak256}(x)$, later reveals x, and once every participant has revealed, the winner is selected

as index $(\sum x_i) \bmod n$ into the array of revealers, receiving an allowance of 9 000. Because this contract's logic is largely independent from marriage and pooling, I used a separate harness, `LotteryEchidnaTest`.

A word on how the harness observes the lottery, since several invariants depend on it. Echidna drives thin wrappers (`startLotteryWrapper`, `joinWrapper`, `revealWrapper`, `endLotteryWrapper`) that forward to the real functions inside a `try/catch`, so a call that reverts *legitimately* – committing before the lottery has started, revealing before the reveal phase, and so on – is swallowed instead of aborting the whole campaign, which lets the fuzzer keep exploring rather than getting stuck on the first ill-timed call. A "win" is detected structurally: `endLotteryWrapper` snapshots every taxpayer's allowance before calling `endLottery()` and compares it afterwards; the taxpayer whose allowance changed is taken as the winner, and its new value and age feed `echidna_winner_gets_correct_prize` and `echidna_winner_is_under_65`. This is deliberately simple, but has one blind spot worth stating up front: a winner whose allowance *already* equalled the prize shows no change and goes unobserved. For the single-round properties here that is a minor gap (an unobserved win is merely not cross-checked); the fairness harness below, where miscounting *would* bias the result, avoids it entirely by recomputing the winner directly rather than inferring it from an allowance change.

Concretely, the win-detecting wrapper and one of the invariants that consumes its result look like this:

```
function endLotteryWrapper() public {
  uint[N] memory before;
  for (uint i = 0; i < N; i++)
    before[i] = taxpayers[i].getTaxAllowance();
  try lottery.endLottery() {
    justEnded = true;
    for (uint i = 0; i < N; i++) {
      uint aft = taxpayers[i].getTaxAllowance();
      if (aft != before[i]) {          // this taxpayer just won
        sawAWin = true;
        lastWinnerAllowance = aft;
        lastWinnerAge = taxpayers[i].getAge();
      }
    }
  } catch { }
}

function echidna_winner_is_under_65() public view returns (bool) {
  if (!sawAWin) return true; // no win observed yet: nothing to check
  return lastWinnerAge < 65;
}
```

The wrapper ends a round and records, in contract-level fields, whether a win happened together with the winner's resulting allowance and age; each `echidna_` invariant then simply reads those fields. The `if (!sawAWin) return true` guard makes the property *vacuously* true until an actual win has been observed, so the invariant constrains only real winners instead of failing on rounds where nobody won.

I wrote one invariant for each of the following properties, and ran Echidna against the unmodified contract without fixing anything first:

- `echidna_winner_gets_correct_prize` – checks that, whenever a win is observed (a taxpayer's allowance changes after `endLottery()`), the new value is exactly 9 000.
- `echidna_no_illegitimate_participants` – a wrapper simulating a non-`Taxpayer` contract calling `commit()` directly; the invariant asserts this must never succeed.

- `echidna_winner_is_under_65` – checks the age of whoever is observed to win.
- `echidna_state_reset_after_round` – checks that `revealed[]` is empty immediately after a successful `endLottery()` call.
- `echidna_no_duplicate_reveals` – checks that no address appears twice in `revealed[]`.

Four of the five failed in a single 50 000-call campaign.

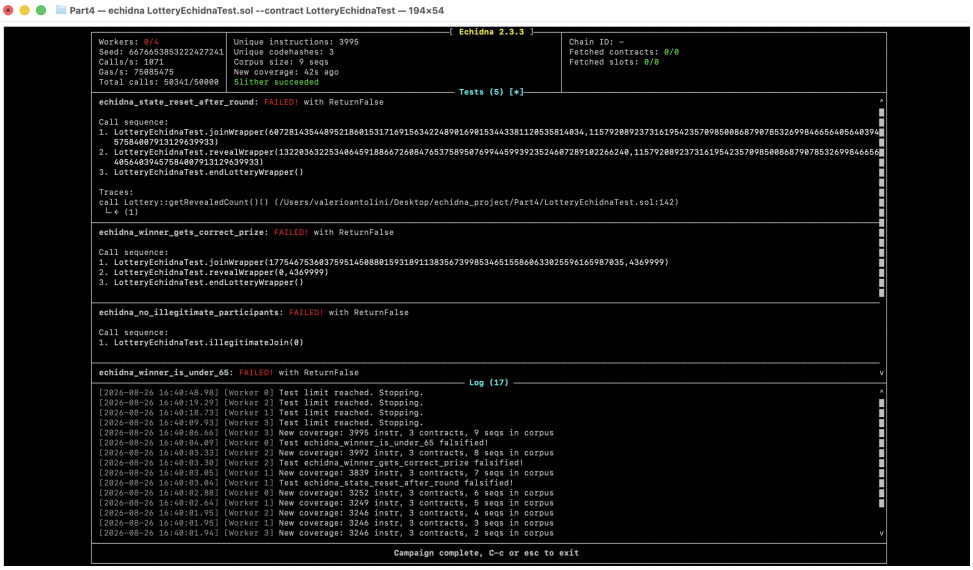


Figure 8: Round 1: state not reset between rounds, wrong prize, and unvalidated participants.

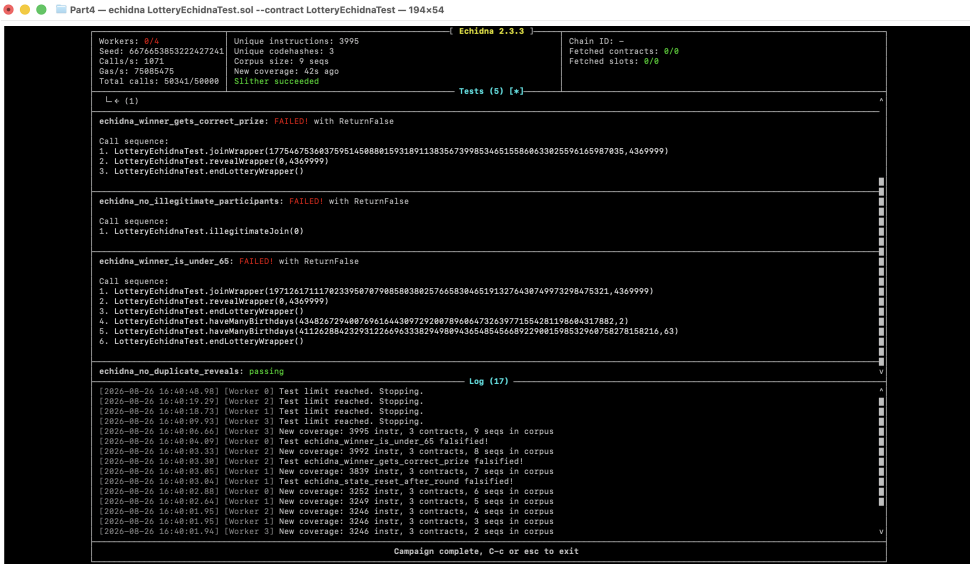


Figure 9: Round 1 (continued): the winner-under-65 property also falsified; only the duplicate-reveal check passed.

The fifth, `echidna_no_duplicate_reveals`, reported `passing` – but code inspection shows `reveal()` has no guard against the same address revealing more than once, which would let a participant appear multiple times in `revealed[]` and have their value counted multiple times in the winner-selection sum. I treat this as a fuzzing *false negative* (the campaign simply hadn't tried that specific sequence yet), not as evidence of correctness, and fixed it in round 2 regardless.

Fixes (round 2). I corrected the prize constant to 9 000; added participant validation and an age check to `commit()`; added a `hasRevealed` guard to `reveal()`; and, for the state-reset issue, tracked every address that committed in the current round (not just the ones who revealed) so that `endLottery()` can fully clear `commits`, `reveals` and `hasRevealed` for all of them – otherwise a participant who committed but never revealed could replay a stale commitment into a future round without recommitting.

```
for (uint i = 0; i < committedAddrs.length; i++) {
  delete commits[committedAddrs[i]];
  delete reveals[committedAddrs[i]];
  delete hasRevealed[committedAddrs[i]];
  delete hasCommittedThisRound[committedAddrs[i]];
}
delete committedAddrs;
delete revealed;
```

Before re-running, I added a sixth, deliberately unaddressed invariant, `echidna_no_late_commits`, targeting a bug noticed during code review but not yet exercised: `commit()` had no upper time bound, so a participant could commit *after* the reveal phase had begun – i.e. after seeing other participants' revealed values on-chain – and choose a value to bias the outcome in their favour.

As expected, `echidna_no_late_commits` failed. More interestingly, `echidna_winner_is_under_65` failed again, despite the age check added in round 1. The minimized call sequence explains why: a taxpayer joined the lottery while still under 65 (passing the check at `commit()` time), then called `haveManyBirthdays` enough times to cross the 65-year threshold *before* revealing. Checking eligibility only once, at commit time, is a classic time-of-check-to-time-of-use (TOCTOU) gap: the participant's age at the moment they actually enter the winner pool (reveal time) can differ from their age when they joined.

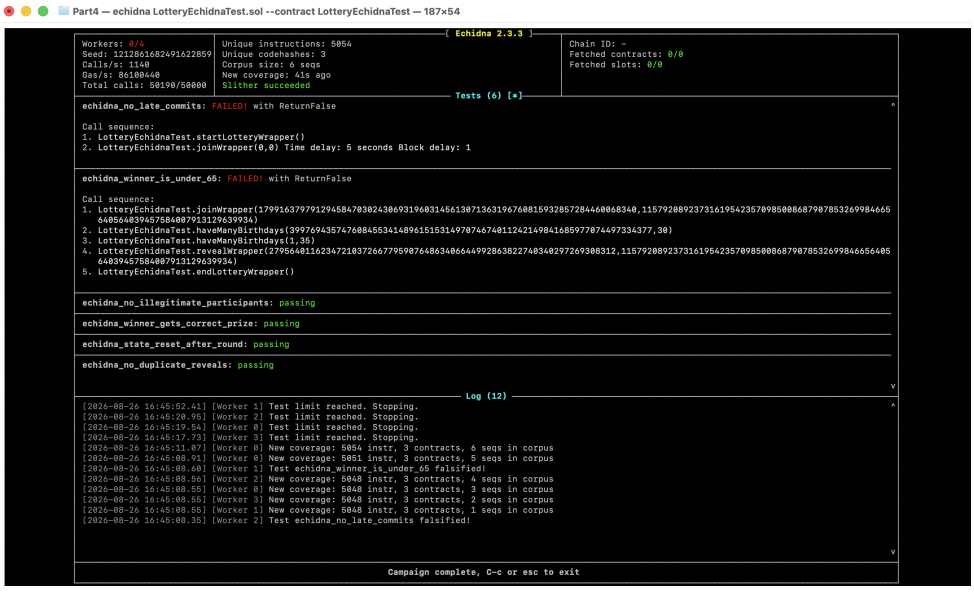


Figure 10: Round 2: the late-commit property fails as expected, and a TOCTOU age bug is found unprompted.

Fixes (round 3). I added an upper bound to `commit()` (`require(block.timestamp < revealTime)`), and re-checked the age requirement inside `reveal()` – the last user-controlled step before a participant enters the winner pool:

```
function reveal(uint256 rev) public {
  require(block.timestamp >= revealTime);
  require(Taxpayer(msg.sender).getAge() < 65, "no longer eligible: 65 or
      older");
  require(!hasRevealed[msg.sender], "you have already revealed this round");
  require(keccak256(abi.encode(rev))==commits[msg.sender]);
  revealed.push(msg.sender);
  reveals[msg.sender] = uint(rev);
  hasRevealed[msg.sender] = true;
}
```

All six invariants then passed a full campaign.

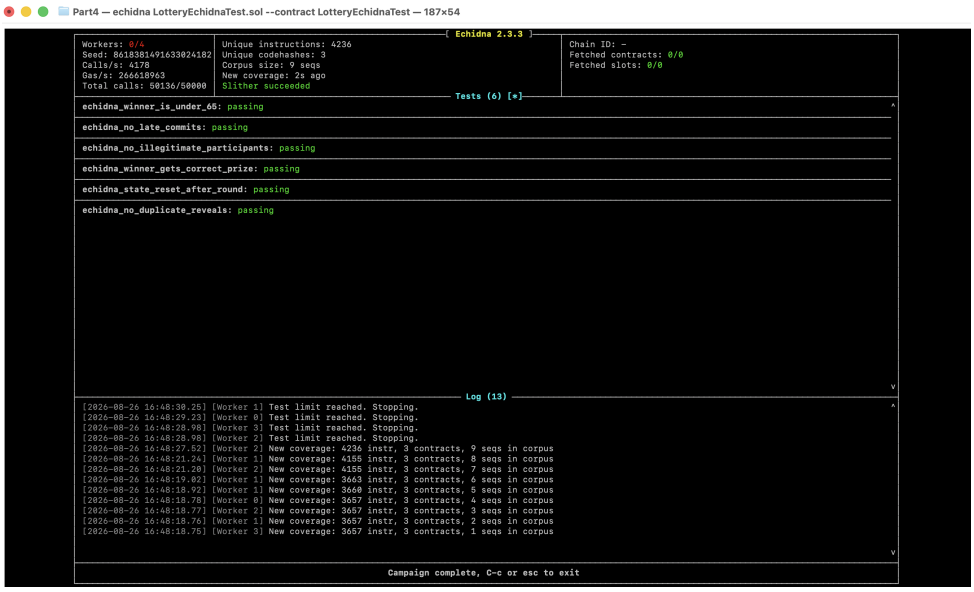


Figure 11: All six invariants passing after round 3.

5.1 Statistical fairness, in a separate contract

The assignment additionally asks that, over many repeated lotteries with the same participants, each participant should win roughly equally often; I keep this counting in a *separate* contract from `Lottery.sol` itself. I wrote `LotteryFairnessTest`, which repeatedly plays full rounds with the same three fixed `Taxpayer` instances, records a per-participant win count, and checks:

```
function echidna_wins_roughly_balanced() public view returns (bool) {
    if (completedRounds < 15) return true; // not enough data yet
    uint maxWins = winCount[0];
    uint minWins = winCount[0];
    for (uint i = 1; i < N; i++) {
        if (winCount[i] > maxWins) maxWins = winCount[i];
        if (winCount[i] < minWins) minWins = winCount[i];
    }
    uint slack = completedRounds / 3;
    if (slack < 3) slack = 3;
    return (maxWins - minWins) <= slack;
}
```

To make repeated rounds tractable for the fuzzer, the harness stores the exact secret used at commit time and replays it automatically in `revealAll()`, rather than relying on Echidna independently fuzzing two matching 256-bit values for commit and reveal. Running this harness

with an increased test limit (100 000 calls) produced over 100 completed rounds without ever violating the balance check.

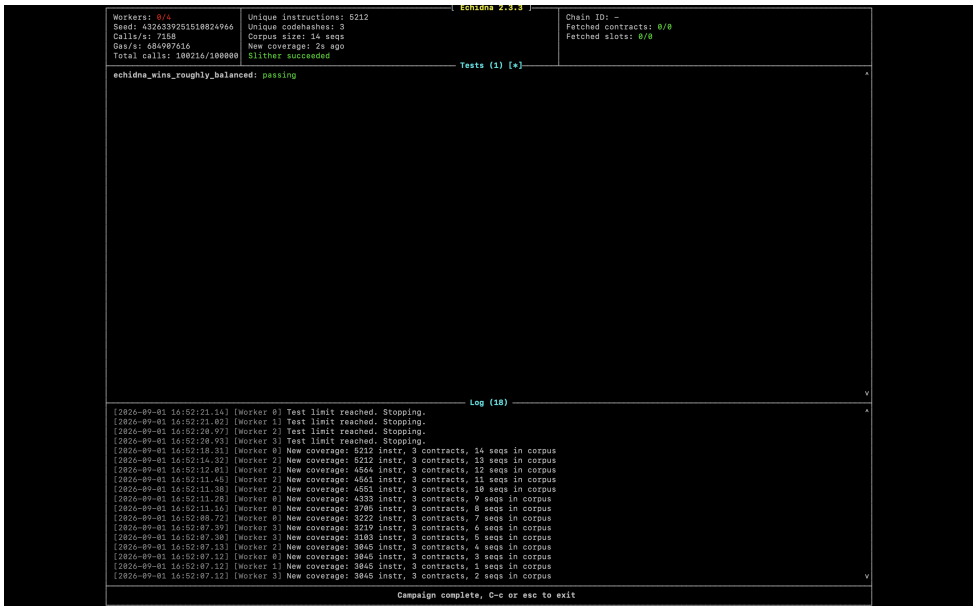


Figure 12: Fairness harness: win counts stay balanced across a large number of completed rounds.

I want to be explicit about what this test does and does not show. Echidna's property mode can only evaluate a deterministic boolean threshold, not compute a rigorous statistic (e.g. a chi-squared test with a p-value); "roughly the same", from the assignment text, is here operationalised as "the gap between the most- and least-frequent winner stays within a generous, round-count-dependent slack". I consider this a reasonable and honest way to test this property with the tool the assignment requires, but it is a heuristic check, not a formal fairness proof.

One point about the counting is worth making explicit, because it is a trap I deliberately avoided. Recording a win by observing that a participant's allowance *changed* at `endLottery()` – the simple approach used by the single-round Part 4 harness – would be wrong here: a winner's allowance is set to the prize and never falls back (these fixed participants never divorce or reset), so a participant winning a *second* time, already at the prize value, would show no change and go uncounted, silently biasing the balance check. `LotteryFairnessTest` therefore recomputes the winner exactly as the contract does – winner = revealed$[(\sum x_i) \bmod n]$ – *before* ending each round, reconstructing each revealer's x_i from the secret the harness stored at commit time. Every win, including repeat wins by the same participant, is thus counted, so the balanced result above is not an artefact of the counting method.

6 Limitations and concluding remarks

A few limitations are worth stating explicitly, in the same spirit as the fairness discussion above:

- **Fuzzing is evidence, not proof.** A campaign that ends with every invariant `passing` means Echidna could not find a counterexample within the calls it tried – not that none exists. Part 4's `echidna_no_duplicate_reveals` false negative in round 1 is a concrete example of this from my own testing.
- **Some access-control weaknesses were fixed by pattern-matching, not by re-discovery.** After finding the `isContract()`-only authorization bug in Part 1 (`setSpouse`) and Part 2 (`setTaxAllowance`), I proactively tightened the identical pat-

tern in `clearSpouse()` (Part 3) instead of waiting for a third, separate campaign to find it again. I believe this is good engineering practice, but it does mean not every fix in this report was independently confirmed as necessary by a failing Echidna run before being made.

- **Timing-dependent properties are harder to fuzz thoroughly.** Echidna advances `block.timestamp` pseudo-randomly between calls; I chose small lottery periods (5 seconds) specifically to make it easier for the fuzzer to cross phase boundaries within a bounded number of transactions, which is a testing simplification rather than a property of the deployed contract.
- **The remaining known weak point is `setSpouse()`/`marry()`'s trust model.** Marriage in this system requires no consent step from the party being married to (see the code comment left in `setSpouse()`); I judged this out of scope for the specific properties the assignment asks for, but it would be the natural next property to test.

Overall, across all four parts I found and fixed ten distinct bugs: two in marriage handling, two in allowance pooling/authorization, and six in the lottery contract (wrong prize, missing participant validation, missing age check, a TOCTOU on that same age check, a missing state reset between rounds, and a double-reveal issue – the last found by code review rather than by a failing Echidna run). The iterative, round-based workflow – write invariants, fuzz the current code, fix the smallest thing that explains the failure, repeat – consistently surfaced real, exploitable issues that would have been easy to miss by code review alone, particularly the multi-transaction bugs (forced remarriage, the TOCTOU age check) that only became visible once Echidna combined several calls in sequence.

A Full Echidna test harnesses

For completeness, this appendix lists the complete source of the three Echidna harness contracts referenced throughout the report, so that *every* invariant and wrapper can be read in full (the body above shows and explains only a representative subset). The contracts under test, `Taxpayer.sol` and `Lottery.sol`, are omitted here since their relevant fixed portions are already shown inline in the corresponding sections.

A.1 TaxpayerEchidnaTest.sol (Parts 1–3)

```
pragma solidity ^0.8.22;

import "Taxpayer.sol";

/// @notice Echidna harness for Part 1 + Part 2 + Part 3.
/// Part 1/2 invariants are kept as regression checks. Part 3 replaces
/// the flat-constant allowance invariants from Part 2 with
/// age-aware versions, since the "correct" default now depends on
/// whether each taxpayer is 65+.
contract TaxpayerEchidnaTest {

    uint constant N = 3; // small population: enough combinations, small state space
    Taxpayer[N] taxpayers;

    constructor() {
        for (uint i = 0; i < N; i++) {
            taxpayers[i] = new Taxpayer(address(0), address(0));
        }
    }

    /// @dev See Part 2 note: lets this harness pose as "a contract" to
```

```
    /// demonstrate the exploit path a malicious external contract
    /// would take against the pre-fix setTaxAllowance().
    function isContract() public pure returns (bool) {
        return true;
    }

    // Wrapper functions (Part 1 + Part 2)

    function marry(uint i, uint j) public {
        Taxpayer(taxpayers[i % N]).marry(address(taxpayers[j % N]));
    }

    function divorce(uint i) public {
        taxpayers[i % N].divorce();
    }

    function haveBirthday(uint i) public {
        taxpayers[i % N].haveBirthday();
    }

    function transferAllowance(uint i, uint change) public {
        taxpayers[i % N].transferAllowance(change);
    }

    function maliciousSetAllowance(uint i, uint value) public {
        taxpayers[i % N].setTaxAllowance(value);
    }

    // Wrapper functions (Part 3)

    /// Fast-forward aging: without this, Echidna would need dozens of
    /// consecutive haveBirthday() calls on the same index just to
    /// reach 65, making the OAP branch and its interaction with
    /// marriage/divorce/pooling unlikely to be explored within the
    /// call budget. Capped to keep gas per call bounded.
    function haveManyBirthdays(uint i, uint8 times) public {
        uint8 t = times % 80;
        for (uint8 k = 0; k < t; k++) {
            taxpayers[i % N].haveBirthday();
        }
    }

    // Helper (not a wrapper): expected allowance for a given index,
    // computed the same way the contract itself should compute it.

    function expectedAllowance(uint idx) internal view returns (uint) {
        uint age = taxpayers[idx].getAge();
        return age >= 65 ? taxpayers[idx].getOapAllowance() : taxpayers[idx].
            getDefaultAllowance();
    }

    // Invariants (Part 1) -- kept as regression checks

    function echidna_marriage_is_symmetric() public view returns (bool) {
        for (uint i = 0; i < N; i++) {
            address spouseAddr = taxpayers[i].getSpouse();
            if (spouseAddr == address(0)) continue;

            bool foundInPopulation = false;
            for (uint j = 0; j < N; j++) {
                if (address(taxpayers[j]) == spouseAddr) {
                    foundInPopulation = true;
```

```
                    if (taxpayers[j].getSpouse() != address(taxpayers[i]))
                        return false;
                    if (!taxpayers[j].getIsMarried()) return false;
                }
            }
            if (!foundInPopulation) return false;
        }
        return true;
    }

    function echidna_married_flag_consistent_with_spouse() public view
        returns (bool) {
        for (uint i = 0; i < N; i++) {
            bool married = taxpayers[i].getIsMarried();
            address spouseAddr = taxpayers[i].getSpouse();
            if (married && spouseAddr == address(0)) return false;
            if (!married && spouseAddr != address(0)) return false;
        }
        return true;
    }

    function echidna_no_self_marriage() public view returns (bool) {
        for (uint i = 0; i < N; i++) {
            if (taxpayers[i].getSpouse() == address(taxpayers[i])) return
                false;
        }
        return true;
    }

    // Invariants (Part 3) -- replace the flat-constant Part 2 versions

    /// Pooling must preserve the *sum* of the two spouses' allowances,
    /// where each spouse's own baseline now depends on their age.
    function echidna_pooled_allowance_sum_correct() public view returns (bool
        ) {
        for (uint i = 0; i < N; i++) {
            if (!taxpayers[i].getIsMarried()) continue;
            address spouseAddr = taxpayers[i].getSpouse();
            for (uint j = 0; j < N; j++) {
                if (address(taxpayers[j]) == spouseAddr && j > i) {
                    uint sum = taxpayers[i].getTaxAllowance() + taxpayers[j].
                        getTaxAllowance();
                    uint expectedSum = expectedAllowance(i) +
                        expectedAllowance(j);
                    if (sum != expectedSum) return false;
                }
            }
        }
        return true;
    }

    /// An unmarried taxpayer must always sit exactly at their own
    /// age-appropriate default (5000 under 65, 7000 at/after 65).
    function echidna_unmarried_allowance_correct() public view returns (bool)
         {
        for (uint i = 0; i < N; i++) {
            if (!taxpayers[i].getIsMarried()) {
                if (taxpayers[i].getTaxAllowance() != expectedAllowance(i))
                    return false;
            }
        }
        return true;
    }
```

```
}
```

Note: the Part 2 harness is identical except that its two allowance invariants (`echidna_pooled_allowance_sum_preserved`, `echidna_unmarried_has_default_allowance`) use the flat 5 000 constant instead of the age-aware `expectedAllowance` helper; the Part 2 version of the first is the snippet shown in Section 3.

A.2 LotteryEchidnaTest.sol (Part 4)

```
pragma solidity ^0.8.22;

import "Taxpayer.sol";
import "Lottery.sol";

/// @notice Echidna harness for Part 4, round 2.
/// Round 1 found: wrong prize (7000 vs 9000), missing participant
/// validation (crash risk), missing age check, and stale state across
/// rounds. All four are fixed in this round's Lottery.sol. We also
/// fix the double-reveal issue even though round 1's campaign didn't
/// happen to trigger it (a fuzzing "miss" isn't proof of correctness --
/// we already know from code review the bug exists).
/// One new invariant is added and deliberately left unfixed:
/// echidna_no_late_commits, targeting the missing upper time bound on
/// commit(). Kept separate from the Part 1/2/3 harness: marriage/pooling
/// invariants and lottery invariants would otherwise conflict (winning
/// the lottery legitimately overrides a taxpayer's allowance from an
/// external source, breaking the "sum preserved between spouses"
/// invariant by design).
contract LotteryEchidnaTest {

    uint constant N = 3;
    Taxpayer[N] taxpayers;
    Lottery lottery;

    // small period so Echidna's natural per-tx time advances are
    // enough to cross commit/reveal/end phase boundaries within a
    // reasonable number of transactions
    constructor() {
        lottery = new Lottery(5);
        for (uint i = 0; i < N; i++) {
            taxpayers[i] = new Taxpayer(address(0), address(0));
        }
    }

    // Bookkeeping for invariants that need "what just happened"

    bool illegitimateCommitSucceeded;
    bool justEnded;
    bool sawAWin;
    uint lastWinnerAllowance;
    uint lastWinnerAge;
    bool committedAfterRevealPhaseBegan;

    // Legitimate flow wrappers (go through Taxpayer, as intended)

    function startLotteryWrapper() public {
        justEnded = false;
        lottery.startLottery();
    }

    function joinWrapper(uint i, uint256 secret) public {
        justEnded = false;
        uint256 rt = lottery.getRevealTime();
```

```
        bool revealPhaseAlreadyStarted = (rt != 0 && block.timestamp >= rt);
        try taxpayers[i % N].joinLottery(address(lottery), secret) {
            if (revealPhaseAlreadyStarted) {
                committedAfterRevealPhaseBegan = true;
            }
        } catch {
            // fine: e.g. reverted because startLottery() hasn't run yet
        }
    }

    function revealWrapper(uint i, uint256 secret) public {
        justEnded = false;
        taxpayers[i % N].revealLottery(address(lottery), secret);
    }

    function haveManyBirthdays(uint i, uint8 times) public {
        uint8 t = times % 80;
        for (uint8 k = 0; k < t; k++) {
            taxpayers[i % N].haveBirthday();
        }
    }

    function endLotteryWrapper() public {
        uint[N] memory before;
        for (uint i = 0; i < N; i++) before[i] = taxpayers[i].getTaxAllowance
            ();

        try lottery.endLottery() {
            justEnded = true;
            for (uint i = 0; i < N; i++) {
                uint aft = taxpayers[i].getTaxAllowance();
                if (aft != before[i]) {
                    sawAWin = true;
                    lastWinnerAllowance = aft;
                    lastWinnerAge = taxpayers[i].getAge();
                }
            }
        } catch {
            // a genuine crash on a fully-legitimate round would need a
            // dedicated "endLottery never reverts on a clean round"
            // property; here we focus on the illegitimate path below.
        }
    }

    // Illegitimate-participant wrapper: simulates a malicious contract
    // that is NOT a real Taxpayer trying to join directly, bypassing
    // Taxpayer.joinLottery() entirely.

    function illegitimateJoin(uint256 secret) public {
        try lottery.commit(keccak256(abi.encode(secret))) {
            illegitimateCommitSucceeded = true;
        } catch {
            // expected: this harness is not a valid Taxpayer contract
        }
    }

    // Invariants

    /// Bug candidate #2: commit() should reject non-Taxpayer callers.
    function echidna_no_illegitimate_participants() public view returns (bool
        ) {
        return !illegitimateCommitSucceeded;
    }
```

```
116
117     /// Bug candidate #4: the same address must not appear twice in
118     /// revealed[] within a round (double-reveal bias).
119     function echidna_no_duplicate_reveals() public view returns (bool) {
120         uint count = lottery.getRevealedCount();
121         for (uint i = 0; i < count; i++) {
122             for (uint j = i + 1; j < count; j++) {
123                 if (lottery.getRevealedAt(i) == lottery.getRevealedAt(j))
                        return false;
124             }
125         }
126         return true;
127     }
128
129     /// Bug candidate #1: the prize must be exactly 9000, not 7000.
130     function echidna_winner_gets_correct_prize() public view returns (bool) {
131         if (!sawAWin) return true; // vacuously true until a win is observed
132         return lastWinnerAllowance == 9000;
133     }
134
135     /// Bug candidate #3: the lottery is for under-65 taxpayers only.
136     function echidna_winner_is_under_65() public view returns (bool) {
137         if (!sawAWin) return true;
138         return lastWinnerAge < 65;
139     }
140
141     /// Bug candidate #5: after a round ends, revealed[] must be empty
142     /// before the next round can start.
143     function echidna_state_reset_after_round() public view returns (bool) {
144         if (!justEnded) return true;
145         return lottery.getRevealedCount() == 0;
146     }
147
148     /// Bug candidate #6 (round 3 target, left unfixed for now): commit()
149     /// has no upper time bound -- a participant could commit after the
150     /// reveal phase has already begun, i.e. after seeing others' values.
151     function echidna_no_late_commits() public view returns (bool) {
152         return !committedAfterRevealPhaseBegan;
153     }
154 }
```

A.3 LotteryFairnessTest.sol (Part 4, statistical fairness)

```
1  pragma solidity ^0.8.22;
2
3  import "Taxpayer.sol";
4  import "Lottery.sol";
5
6  /// @notice Separate harness that repeatedly plays many full lottery
7  /// rounds with the SAME fixed set of 3 taxpayers, and checks that wins
8  /// end up roughly balanced across them -- the statistical fairness
9  /// property from the assignment text, as opposed to the single-round
10 /// correctness properties already covered in LotteryEchidnaTest.sol.
11 contract LotteryFairnessTest {
12
13     uint constant N = 3;
14     Taxpayer[N] taxpayers;
15     Lottery lottery;
16
17     uint256[N] lastSecret; // remembers what each participant committed,
18                            // so revealAll() can replay the exact same
19                            // value instead of relying on Echidna
20                            // fuzzing two matching values independently
```

```
21
22      uint[N] winCount;
23      uint completedRounds;
24
25      constructor() {
26          lottery = new Lottery(5);
27          for (uint i = 0; i < N; i++) {
28              taxpayers[i] = new Taxpayer(address(0), address(0));
29          }
30      }
31
32      // Wrappers -- deliberately simple/deterministic: here we want
33      // *many completed rounds*, not edge-case exploration.
34
35      function startRound() public {
36          try lottery.startLottery() {} catch {}
37      }
38
39      function commitAll(uint256 s0, uint256 s1, uint256 s2) public {
40          lastSecret[0] = s0;
41          lastSecret[1] = s1;
42          lastSecret[2] = s2;
43          try taxpayers[0].joinLottery(address(lottery), s0) {} catch {}
44          try taxpayers[1].joinLottery(address(lottery), s1) {} catch {}
45          try taxpayers[2].joinLottery(address(lottery), s2) {} catch {}
46      }
47
48      function revealAll() public {
49          try taxpayers[0].revealLottery(address(lottery), lastSecret[0]) {}
                catch {}
50          try taxpayers[1].revealLottery(address(lottery), lastSecret[1]) {}
                catch {}
51          try taxpayers[2].revealLottery(address(lottery), lastSecret[2]) {}
                catch {}
52      }
53
54      function endRound() public {
55          // Compute the winner exactly as Lottery.endLottery() does --
56          // winner = revealed[(sum of revealed secrets) % n] -- BEFORE
57          // ending the round, instead of inferring it from an allowance
58          // change afterwards. This makes repeat wins countable: a winner
59          // already at the prize value shows no allowance change, so a
60          // naive before/after comparison would miss it and bias counts.
61          uint revealedCount = lottery.getRevealedCount();
62          uint winner = N; // sentinel: no valid winner this round
63          if (revealedCount > 0) {
64              uint total = 0;
65              for (uint k = 0; k < revealedCount; k++) {
66                  total += secretOf(lottery.getRevealedAt(k));
67              }
68              winner = indexOf(lottery.getRevealedAt(total % revealedCount));
69          }
70
71          try lottery.endLottery() {
72              completedRounds++;
73              if (winner < N) winCount[winner]++;
74          } catch {}
75      }
76
77      // Maps a revealer's address back to the secret it committed this
78      // round (== the value stored in Lottery.reveals for that address).
79      function secretOf(address a) internal view returns (uint256) {
80          for (uint i = 0; i < N; i++) {
```

```
            if (address(taxpayers[i]) == a) return lastSecret[i];
        }
        return 0;
    }

    // Maps a taxpayer address back to its index, or N if not one of ours.
    function indexOf(address a) internal view returns (uint) {
        for (uint i = 0; i < N; i++) {
            if (address(taxpayers[i]) == a) return i;
        }
        return N;
    }

    // Observability getters, useful when reporting results

    function getCompletedRounds() public view returns (uint) {
        return completedRounds;
    }

    function getWinCount(uint i) public view returns (uint) {
        return winCount[i % N];
    }

    // Invariant

    /// After enough completed rounds, no participant's win count should
    /// be wildly out of line with the others. Slack is intentionally
    /// generous (a rough heuristic, not a precise statistical test) to
    /// avoid flagging on natural variance.
    function echidna_wins_roughly_balanced() public view returns (bool) {
        if (completedRounds < 15) return true; // not enough data yet

        uint maxWins = winCount[0];
        uint minWins = winCount[0];
        for (uint i = 1; i < N; i++) {
            if (winCount[i] > maxWins) maxWins = winCount[i];
            if (winCount[i] < minWins) minWins = winCount[i];
        }

        uint slack = completedRounds / 3;
        if (slack < 3) slack = 3;
        return (maxWins - minWins) <= slack;
    }
}
```